从3632个漏洞看AI时代的评测基准重构！VulnGym基准发布

原创 腾讯程序员 腾讯技术工程 May 26, 2026, 2:32 AM 广东 9 listened



作者：腾讯悟空代码安全团队

导语

> 随着 Claude Security、Codex Security 等新一代漏洞检测工具出现，漏洞检测正在从"发现代码缺陷"走向"理解项目上下文并检测业务风险"。

我们分析了 2025 年至今的 3,632 条漏洞报告，发现**业务逻辑漏洞占比明显上升**。尤其是在 GitHub Star 数超过 1 万的热门项目中，4 月份业务逻辑漏洞占比已达到 47.2%，接近一半。

然而，现有评测基准大多还停留在传统漏洞类型上，难以判断安全工具是否能够在真实项目上发现跨文件、跨流程的复杂业务逻辑风险。因此，漏洞检测评测基准需要升级，转向更贴近真实项目和真实攻击路径分布的新一代基准。

为此，腾讯悟空安全团队联合**香港中文大学 ARISE Lab、复旦大学系统软件与安全实验室、香港大学 JC STEM Lab of Intelligent Cybersecurity、北京大学 Narwhal-Lab、中国科学院信息工程研究所网络威胁分析研究室**共同发布 **VulnGym 评测基准**。

VulnGym是一个面向白盒漏洞检测 Agent 的真实项目评测基准，基于 GitHub 高星项目中的真实漏洞构建，覆盖 **400+ 条漏洞路径**，其中 **71.2% 为业务逻辑漏洞**。每个样本都标注漏洞入口、敏感操作和触发路径，支持可复现、可解释的确定性评测。

Github: https://github.com/Tencent/VulnGym

Hugging Face:

https://huggingface.co/datasets/tencent/VulnGym

一、漏洞的形态，正在被 AI 编码悄悄改写

在 2026 年 5 月初，我们针对公开漏洞数据进行了漏洞类型分析与统计，一组反直觉的数据浮现了出来，这表明随着 AI Coding 工具的发展，**漏洞的形态可能正在悄悄发生变化**。

1.1 一次统计带来的反直觉信息

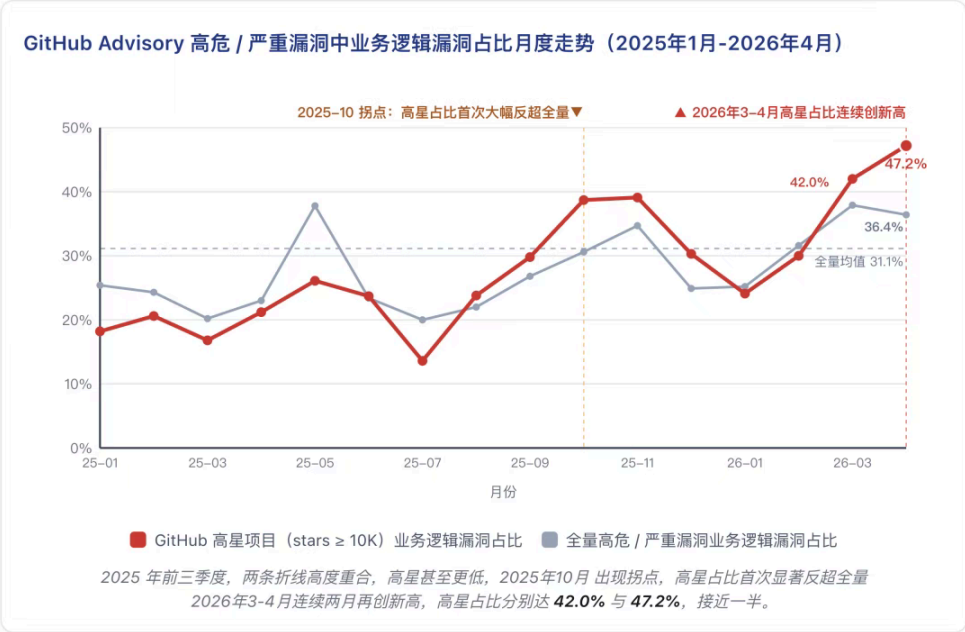
我们的统计范围是 GitHub Advisory (Reviewed) 2025 年 1 月至 2026 年 4 月期间的 **3,632 条高危 / 严重漏洞**（高危 2,768 + 严重 864）。

> 注：
>
> - 由于业界缺乏统一定义，在本文中"业务逻辑漏洞"是指攻击者不违反任何技术规则，仅通过滥用合法功能来达成系统设计者未预期结果的缺陷。
> - "业务逻辑漏洞"的判定为规则化打标 + 人工抽检。该来源不能等同于 CVE 全集，结论仅反映该样本范围内的分布特征。

下面是这些漏洞报告的统计结果：

维度	数据
整体业务逻辑漏洞占比	31.14%（1,131 / 3,632）
2025 全年业务逻辑漏洞占比	25.67%（382 / 1,488）
2026 年 1–4 月业务逻辑漏洞占比	34.93%（749 / 2,144）
2025 全年中 stars ≥ 10K 的高星项目业务逻辑漏洞占比	26.11%（141 / 540）
2026 年 1–4 月中 stars ≥ 10K 的高星项目业务逻辑漏洞占比	39.59%（382 / 965）

上面的整体结果无法很好明确趋势变化，那么将这份数据拆分成月度趋势，可以看出业务逻辑漏洞占比这条曲线还在向上走。



1.2 反直觉的数据引发的二次思考

上面这条曲线呈现出一个反直觉现象：2025 年前三季度，高星项目中的业务逻辑漏洞占比低于整体平均水平（21.4% vs. 24.1%）。但从 **2025 年 10 月**开始，趋势明显反转：过去半年，高星项目中业务逻辑漏洞占比平均超过 38.9%；在 2026 年 3 月和 4 月更是连续创新高，4 月更是达到了 47.2%，接近一半的数量级。

这带来两个核心问题值得我们思考：

- **为什么业务逻辑漏洞占比转折点恰好是 2025 年下半年？**这之前 3 个季度的曲线一直相对平均。
- **为什么在高星项目中业务逻辑漏洞占比更高？**常识里它们应当有更严格的 code review 流程。

为了解释这一变化，下面我们尝试从两个相对合理、且有外部数据支撑的角度展开分析。

1.3 供给侧：AI 正在改变漏洞的产生方式

AI Coding 工具的快速普及，正在改变漏洞的来源。

以 Claude Code 为例，它的月下载量从 **2025 年 6 月的 430 万**，快速增长到 **2025 年 7 月的 1,704 万**，并在 **2026 年 3 月达到 4,444 万**。有意思的是，这两个时间点也正好对应业务逻辑漏洞占比的两次明显上升：**2025 年 10 月后，高星项目的业务逻辑漏洞占比开始持续高于整体**

平均水平；2026 年 3—4 月，这一比例进一步升至 47.2%。



时间上的接近并不能直接证明因果关系，但它提示了一个值得追问的问题：当 AI 开始深度参与代码生成，漏洞的类型是否也在发生变化？

我们可以从下面这三层来看：

- ① **第一层：传统漏洞被自然防御。**LLM 写代码时倾向遵循框架的安全默认值，参数化、转义、类型安全 API 是默认选项。"肌肉记忆型"缺陷在新代码中显著稀释——这是一件好事，但也意味着以这类漏洞为主要测试目标的评测基准开始失去信号。

- ② **第二层：业务逻辑漏洞被放大。**LLM 擅长补齐"看起来合理"的代码，但**不具备对项目特有业务规则的长期持有**：谁是管理员？哪个接口需要鉴权？资源是否多租户隔离？这些规则需要对整个工程项目的跨模块理解。AI Coding 工具在这些维度容易产生**形式正确、业务逻辑存在缺陷**的代码。这种趋势在 AI-native 项目中更明显。我们筛选出的 **36 个头部 AI Agent 项目**中，共有 **500+ 漏洞报告**，其中业务逻辑漏洞占比达到 41.3%，比整体平均水平高出约 10 个百分点。从 2026 年初爆火的 OpenClaw，到后来的 Hermes 等代表性项目，业务逻辑漏洞占比惊人地超过 50%。

- ③ **第三层，AI 生态自带的新漏洞类别。**AI 应用本身还带来了过去很少见的问题，例如 prompt 注入、工具白名单绕过、MCP 能力越权等。这些问题很难被传统 CWE 分类完整覆盖，本质上仍属于业务逻辑漏洞，但表现形式已经是 AI 时代特有的。

换句话说，前两层是"旧漏洞类型的占比在变"，而第三层是"新的漏洞类型正在出现"——这类漏洞目前几乎不在任何主流安全评测基准的覆盖范围之内。

1.4 检测侧：AI 开始发现"过去看不见"的漏洞

另一条同样重要的时间线，发生在漏洞检测工具本身。

过去几个月，Anthropic 的 Project Glasswing、OpenAI 的 Daybreak，以及 Microsoft MDASH、AWS Security Agent 等安全项目密集出现。它们和传统 SAST/DAST 工具最大的不

同在于：**不再只是按规则匹配代码，而是开始理解跨文件、长上下文里的业务语义。**



传统安全工具更擅长发现 SQL 注入、越界访问这类"代码写错了"的问题；但业务逻辑漏洞往往不是代码语法错误，而是**代码看起来没问题，却违背了业务规则**。比如接口本身能正常运行，但不该访问的人也能访问；流程看似完整，但关键权限校验被绕过。

因此，2026 年以来业务逻辑漏洞报告占比继续上升，并不一定只说明"新代码里产生了更多这类漏洞"。更可能的原因是：**过去很难被发现的业务逻辑缺陷，正在被新一代 AI 安全检测系统批量挖出来。**

1.5 小结：漏洞的形态在变，评价的标尺也需要变化

结合上述的数据统计与分析，一个判断目前可以基本被勾勒出轮廓——

> AI 编码时代的高危漏洞画像，正在从「传统编码错误」迁移到「形式正确、业务逻辑缺陷」。这一判断是否构成范式位移，还需要更长窗口的数据继续观察。

但即使按现有数据保守估计，一个直接的问题已经浮出水面：当评测对象的形态正在变化，过去用来做漏洞检测能力评判的标尺，在 AI 时代**还对得上吗？**

二、现有评测基准的系统性失配

现有的漏洞检测能力评测基准具体情况是怎样的呢？

判断一个漏洞检测能力基准是否对准了"项目级业务逻辑漏洞"这条赛道，可以从三个维度来分析：

维度	含义
评测粒度	评测单元是 function / file / project
漏洞类型	以模式化 CWE 为主，还是涵盖业务逻辑
测评形态	白盒（给代码）还是黑盒（给服务）

我们用这三个维度盘点了主流的漏洞检测 benchmark。**每一项独立评估是否满足"项目级业务逻辑漏洞"评测所需的 4 个核心维度**，✅ 表示满足、❌ 表示不满足、⚠️ 表示部分满足：

Benchmark	项目级粒度	白盒输入	业务逻辑漏洞	可验证触发路径	关键差距
OWASP Benchmark (v1.2)	❌	✅	❌	❌	合成 Java servlet，模式注入为主
Juliet Test Suite (SARD)	❌	✅	❌	❌	单文件方法级，缺跨模块语义
Devign	❌	✅	❌	❌	函数级二分类，仅布尔标签
BigVul / D2A / CVEFixes	❌	✅	⚠️	❌	函数 / diff 粒度，偏注入 / 内存
CWE-Bench-Java	⚠️	✅	⚠️	❌	仓库级弱标注，业务逻辑占比低
SecurityEval / LLMSecEval	❌	✅	❌	❌	测生成是否安全，非发现能力
CyberGym / CyberBench	✅	✅	⚠️	⚠️	面向黑盒场景评测，与白盒互补
SWE-Bench / SWE-Bench-Verified	✅	✅	⚠️	❌	非安全基准，GT 为 patch
VulnGym	✅	✅	✅	✅	**本文提出**：均符合

把这些基准映射到三维空间，会发现它们至少在一个维度上偏离了"项目级业务逻辑漏洞"：

- **模式集中、函数级标注**（Devign、BigVul 等传统测评集）：ground truth 是二分类或 diff，漏洞类型压倒性地偏向 SQLi / XSS / 内存安全等传统漏洞——而这恰恰是 LLM 写的代码**越来越少**出现问题的那一类。
- **合成、玩具上下文**（OWASP Benchmark、Juliet）：真实项目需要的跨模块推理无法在合成样例里复现。
- **黑盒、exploit 导向**（CyberGym 等 CTF 类）：测评的是 Agent 对运行服务的攻击能力，而非在源码中**定位缺陷**的能力。这是一条非常有价值的赛道，但**与白盒代码理解是互补关系**，不是替代关系。

> 可以看到除 VulnGym 外，没有任何一个现有基准同时满足「项目级 + 白盒 + 以真实业务逻辑漏洞为主 + 有可验证的触发路径」这 4 个维度。

这就是当前漏洞检测能力评测基准的**系统性失配**——评测的对象在变化，标尺却没有跟上。

三、VulnGym：项目级 + 白盒 + 业务逻辑主导 + 三要素路径

VulnGym 不是凭空设计的——它是顺着上面那条趋势线长出来的。

VulnGym 是一个面向白盒漏洞检测 Agent 的真实项目级评测基准。它以 GitHub Advisory 中 reviewed 报告为数据源，每条样本绑定真实开源仓库的特定漏洞 commit，并提供人工审核过的**漏洞入口（entry point）**、**敏感代码操作（critical operation）**与**跨模块推理链路（trace）**三要素标注，从而支持可复现、可解释的确定性评测。

三个核心设计理念：

- **真实项目级评测单元**— 每个样本绑定到含漏洞的特定版本代码仓库，评测 Agent 在真实多文件、多模块工程中的漏洞发现与定位能力，而不是孤立的函数或合成片段。
- **全面的漏洞类型覆盖**— 既涵盖需要跨模块代码语义理解的**业务逻辑漏洞**（授权绕过、认证缺失、Agent 能力边界绕过等），也包含**传统安全漏洞**（代码 / 命令注入、路径穿越、反序列化等）。
- **可验证的漏洞路径**— entry_point + critical_operation + trace 的三要素位置标注，让评测器可以精确区分"工具蒙对了一个位置"和"工具完整理解了漏洞"。

3.1 数据来源与覆盖：从真实漏洞中筛选，锚定业务逻辑

在讲分布之前，先讲清楚一件事：VulnGym 的样本不是合成的、不是 CTF 题、也不是教科书例子，它是从开源生态中的漏洞数据库——**GitHub Advisory**——按三道筛选条件切出来的。

筛选 ① · 来源权威：GitHub Advisory · Reviewed

仅采用状态为 reviewed 的安全公告——已被 GitHub 安全团队人工审核，且与 CVE 体系打通。→ 排除未经审核、低质量或非安全性问题的噪声。

筛选 ② · 项目真实：高 Star 项目（≥ 10K）

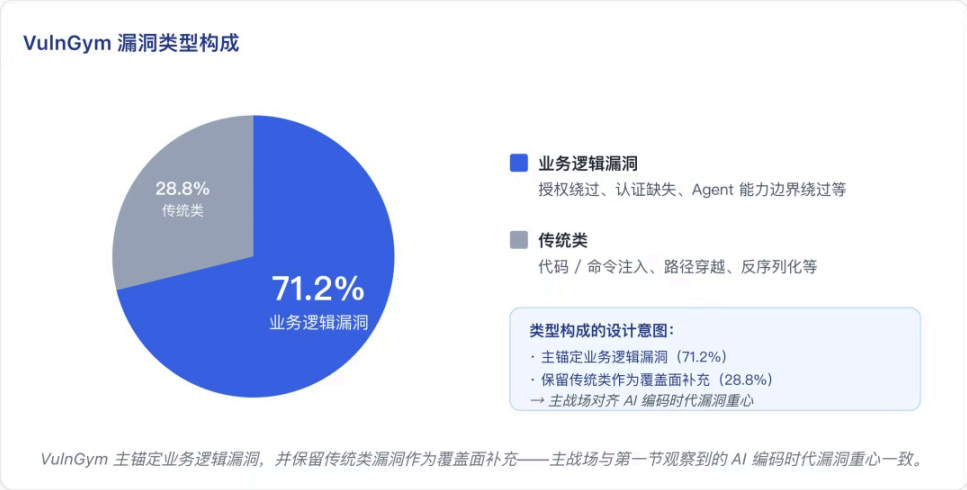
仅保留 GitHub Star ≥ 10K 的开源项目所对应的报告——它们有真实用户、有真实部署、被产业广泛依赖。→ 排除玩具仓库与已废弃项目，确保漏洞源自真实业务场景。

筛选 ③ · 时效与严重性：近半年 · HIGH / CRITICAL

仅保留近半年内披露的 HIGH 与 CRITICAL 等级漏洞。→ 反映 AI 编码时代当下的漏洞形态，而不是历史沉淀。

三道筛选过完之后，从 3,632 条漏洞报告中，我们仅保留了 **184 条漏洞报告**，从报告中分析并提取了 **400+ 漏洞链路数据**。同时 VulnGym 主要锚定业务逻辑漏洞，占比 71.2%。

同时，为了保证整体覆盖面，我们也保留了一定比例的**传统类漏洞**（代码 / 命令注入、路径穿越、反序列化等）作为对照样本与广度补充。



3.2 标注粒度：从「二分类」走向「路径级标注」

标注粒度是 VulnGym 区别于现有基准最关键的工程差异。

传统基准的 ground truth 是一个布尔值（vulnerable / not）或一段 patch diff。两者都无法回答一个核心问题：**Agent 是真的理解了这个漏洞，还是只是蒙对了一个位置？**

VulnGym 的每条 entry 都包含人工审核过的三处位置：

```
entry_point  ──→  trace[0], trace[1], ..., trace[n]  ──→  critical_operation

外部可达入口          跨模块数据 / 控制流链路          敏感代码操作 / 缺陷触发点
```

- **entry_point**：攻击者可直接触达的入口（HTTP handler、gRPC interceptor、前端交互位点、CLI 命令等）。
- **critical_operation**：缺陷真正被触发的那一行代码（innerHTML 赋值、绕过 auth 的条件判断、工具白名单比较、状态机跳过等）。
- trace[]：从 entry_point 到 critical_operation 之间的有序跨文件 / 跨模块步骤，每一步是 `{file, line, code}`。

这是目前同类 benchmark 中最细的标注粒度：

标注层次	传统基准	VulnGym
Ground Truth 形态	vulnerable / not 或 patch diff	entry_point × trace × critical_operation 三要素位置

标注层次	传统基准	VulnGym
评测可解释性	命中 / 未命中	入口、链路、缺陷触发点逐项对齐
是否能区分"蒙对"	否	是
是否支持跨模块推理评测	大多不支持	原生支持

> 让评测可以精确区分「工具蒙对了一个位置」和「工具完整理解了漏洞」——这是 VulnGym 在标注层面的更细粒度的要求。

四、VulnGym 之上：能用它来比较什么

除了直接当数据集用，VulnGym 也可以当成一把统一的尺子，用来横向比较不同的漏洞检测方案——同一批题目，谁找得更全、谁定位得更准，一目了然。

而一套漏洞检测方案，通常涉及三个变量：**底层模型**、**Harness 工程**、**Agent 工具整体**。围绕这三个变量，VulnGym 自然衍生出三种常见用法。

① 比模型：哪个 LLM 更懂业务逻辑漏洞？

固定 Harness 工程，只换底层模型——Claude、GPT、Gemini、DeepSeek、混元等。同一套题、同一套工具，看不同模型谁能多找出几个、错报少多少、长链路推理时谁先掉链子，以及模型版本升级到底带来了多少真实能力提升。

② 比 Harness：怎么"喂"代码给它效果最好？

固定模型，只调工程层——一次给多少代码、要不要先把架构信息塞进去、上下文怎么管理、Skills、MCP 和子 agent 怎样调度。这是离落地最近的一层，往往也是性价比最高的优化空间。

③ 比 Agent：哪种 Agent 工具整体检测更加有效？

Agent 工具整体对比——Claude Security、Codex Security 等各家发布的检测工具和能力。看谁的工具在模型和 harness 上的结合最完善，谁的检测能力真的有用，谁是花架子。



三组对比加起来，VulnGym 让原本很模糊的问题变得可以拿数字回答：

- **现在到了什么水平**：主流方案在真实业务逻辑漏洞上能拿多少分；
- **瓶颈在哪一块**：是模型不行、harness 不行、还是整体适配没调好；
- **每次优化有没有用**：迭代是否真的带来了可度量的提升，而不是"感觉好了一些"。

五、一个起点：邀请大家一起打磨

漏洞检测评测基准是这个行业的"标尺"。

过去十年里，这把标尺被反复打磨——从 OWASP Benchmark 的 SQLi/XSS 题库，到 Devign 的函数级二分类，再到 BigVul / CVEFixes 的 CVE 级 diff，再到 SWE-Bench 的仓库级补丁——每一次刻度的细化，都对应了那一代代码缺陷的真实形态。

而当 AI 深度参与代码构建之后，漏洞的形态可能也在悄悄变化：传统类型漏洞正在被框架与模型的安全默认值天然防御，「与业务逻辑强耦合」的缺陷，开始更频繁地出现在视野里。这是否最终会被确认为一次范式层面的位移，我们自己也仍在持续观察——但至少，**让标尺顺着这条线索更新，是一件值得做的事情**。

VulnGym是我们在这条线索上迈出的第一步，肯定还远谈不上完善：标签体系会有遗漏、个别样本的标注可能存在偏差。我们更愿意把它看作一个起点——后续版本会持续扩展漏洞类型、项目覆盖等，并**配套发布 Leaderboard 与可复现的评测 Harness**。

所以，**我们也非常希望听到你的声音**，欢迎贡献真实漏洞样本、纠正现有标注，并对标签体系、评测方法与 Harness 设计提出建议，帮助 VulnGym 持续校准并更贴近真实世界。

开源链接：

- **GitHub**：https://github.com/Tencent/VulnGym
- **HuggingFace 数据集**：https://huggingface.co/datasets/tencent/VulnGym

同时也欢迎加入悟空安全团队的社群，试用最新版本的悟空 Agent 能力。

> **关于腾讯悟空代码安全团队**
>
> 腾讯悟空代码安全团队（WuKong, Tencent）隶属腾讯安全平台部，专注于 AI 驱动的代码层漏洞检测与安全风险治理。自研悟空代码安全 Agent 已在 GitHub 知名开源项目中累计发现并确认 400+ 0day 漏洞，获微软、英伟达、Apache 等知名企业与开源组织致谢。团队曾发布业内首个项目级 AI 生成代码安全评测集 A.S.E，团队成员过往研究成果发表于 S&P，USNEIX Security，FSE，ASE，ICSE，ACL 等国际顶尖学术会议。





悟空 AI Agent · 目录

〈上一篇 · “龙虾”也需要看病？一张旧病历，引出三个新确诊

阅读 6671 修改于2026年5月28日

留言

写留言

还没有留言，来写首评